ALGORITHM AND FLOWCHART

ALGORITHM

1. What is an algorithm?

An Algorithm is a step by step method for solving problems

2. Characteristics of a good algorithm

- **Input** - takes zero or more values (e.g. *Read n*).
- **Output** - gives at least one result (e.g. *Print sum*).
- **Definiteness** - every step is clear and has only one meaning.
- **Finiteness** - it must stop after a limited number of steps.
- **Effectiveness** - each step is simple enough to be carried out.

3. Rules for writing an algorithm

- Step 1 is always **Start** and the last step is always **Stop**.
- **Initialize** every variable (like i = 1, sum = 0, count = 0) before using it.
- Use **Read** to take input and **Print** to show output.
- Use **Check** for a decision. Write both exits: *if true goto step __, if false goto step __*.
- A loop is made with **goto**: after the last step of the loop body, goto the Check step again.

4. Keywords used

Keyword	Meaning	Example
Start / Stop	Beginning and end of the algorithm	Step 1: Start
Initialize	Give starting values to variables	Initialize i = 1, sum = 0
Read	Take input from the user	Read n
Print	Display an output	Print sum
Check	Test a condition (true / false)	Check i <= n
goto	Jump to another step (used for loops)	goto step 4

ALGORITHM AND FLOWCHART

5. Operators used

Operator	Meaning	Example
=	Assign a value	sum = sum + i
==	Equal to (comparison)	n % i == 0
< <= > >=	Less / greater than (or equal)	i <= n
%	Remainder (modulus)	6 % 4 = 2, 253 % 10 = 3
/	Division (integer division for digits)	253 / 10 = 25
* ^	Multiply, power	f * i, d^c

6. Example - algorithm to add two numbers

Step 1: Start

Step 2: Read a, b

Step 3: sum = a + b

Step 4: Print sum

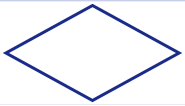
Step 5: Stop

FLOWCHART · Symbols and rules

1. What is a flowchart?

A flowchart is the picture of an algorithm. It uses standard symbols joined by arrows to show the order in which the steps are carried out. Each step of the algorithm becomes one symbol.

2. Flowchart symbols

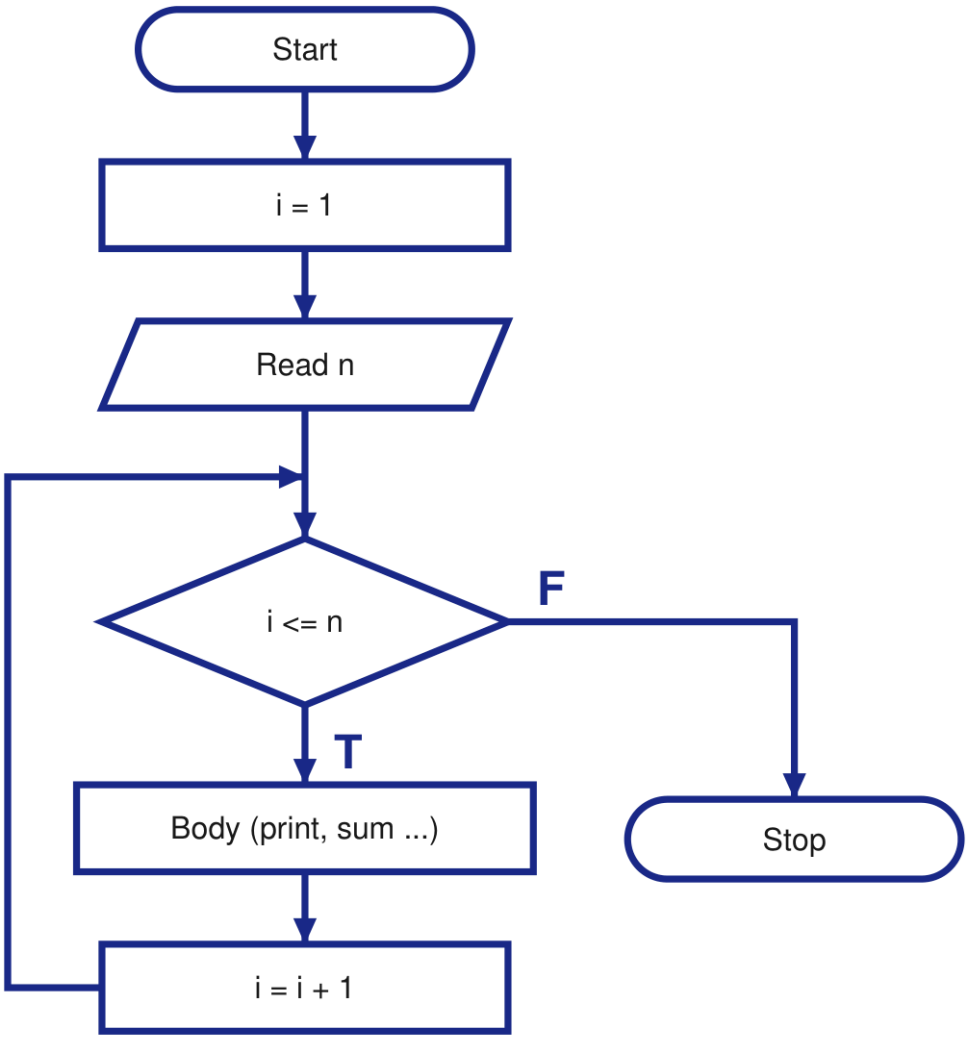
Symbol	Name	Used for	Example
	**Terminal (Oval)**	Start and Stop of the flowchart	Start, Stop
	**Input / Output (Parallelogram)**	Reading input and printing output	Read n, Print i
	**Process (Rectangle)**	Calculations, assignments, initialization	sum = sum + i
	**Decision (Diamond)**	A condition with two exits: T (true) and F (false)	i <= n
	**Flow line (Arrow)**	Shows the direction of flow from one symbol to the next	
	**Connector (Circle)**	Joins two parts of a flowchart (or two pages)	A

3. Rules for drawing a flowchart

- Every flowchart has exactly one Start and one Stop oval.
- Draw from top to bottom; use arrows on every line.
- A decision diamond has one entry and two exits, labelled T (true) and F (false).
- Use a parallelogram for Read / Print and a rectangle for calculations.
- For a loop, draw an arrow from the end of the loop body back to the line just above the decision diamond.
- Keep symbols the same size and write short, clear text inside them.

FLOWCHART · Symbols and rules

4. How a loop looks in a flowchart



Initialize: sets i = 1 before the loop

Condition: i <= n is checked every time

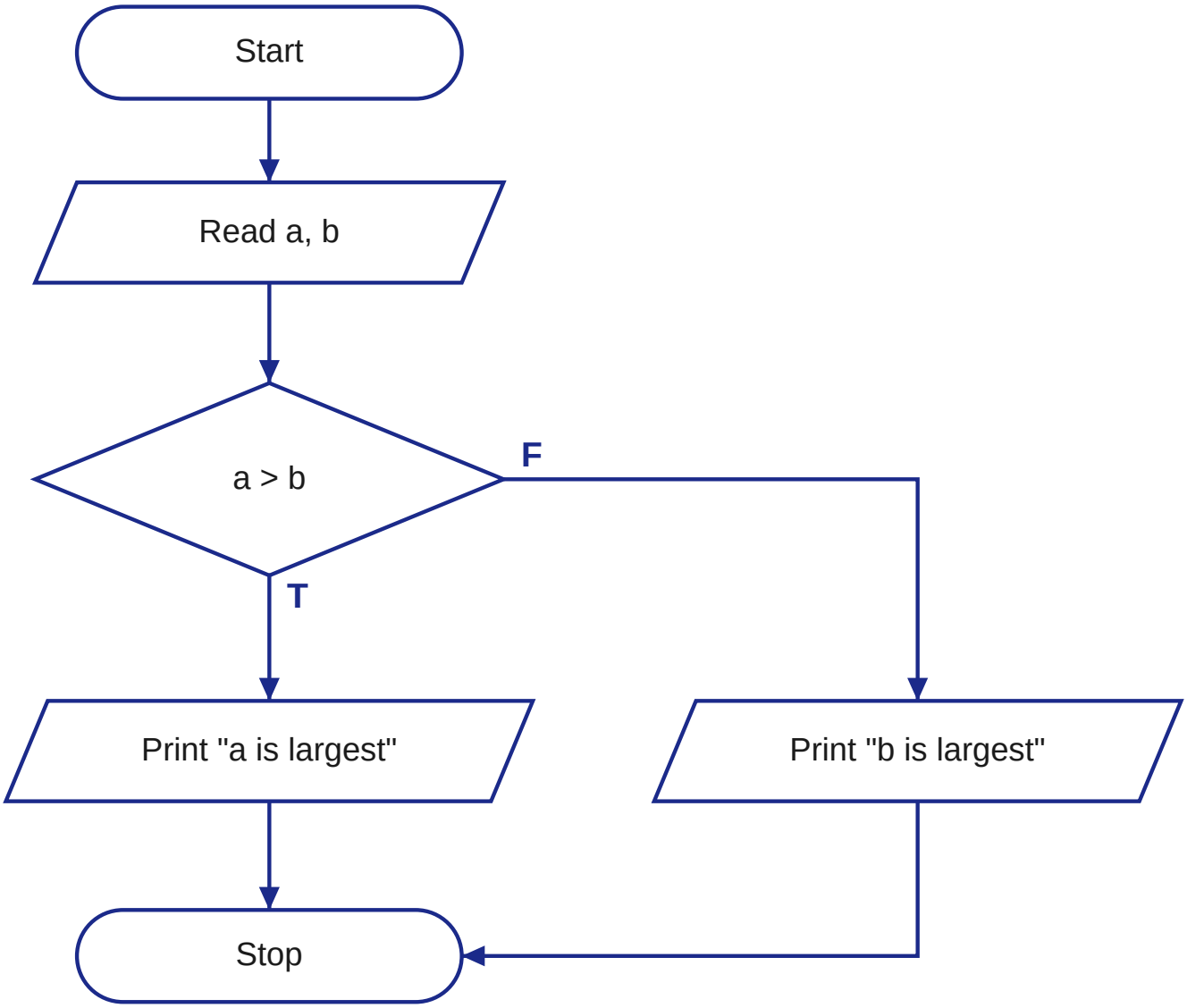
T path: body runs while condition is true

Update: i = i + 1 moves to the next value

Loop back: arrow returns above the diamond

F path: loop ends, goes to Stop / output

Q1. Largest of 2 numbers

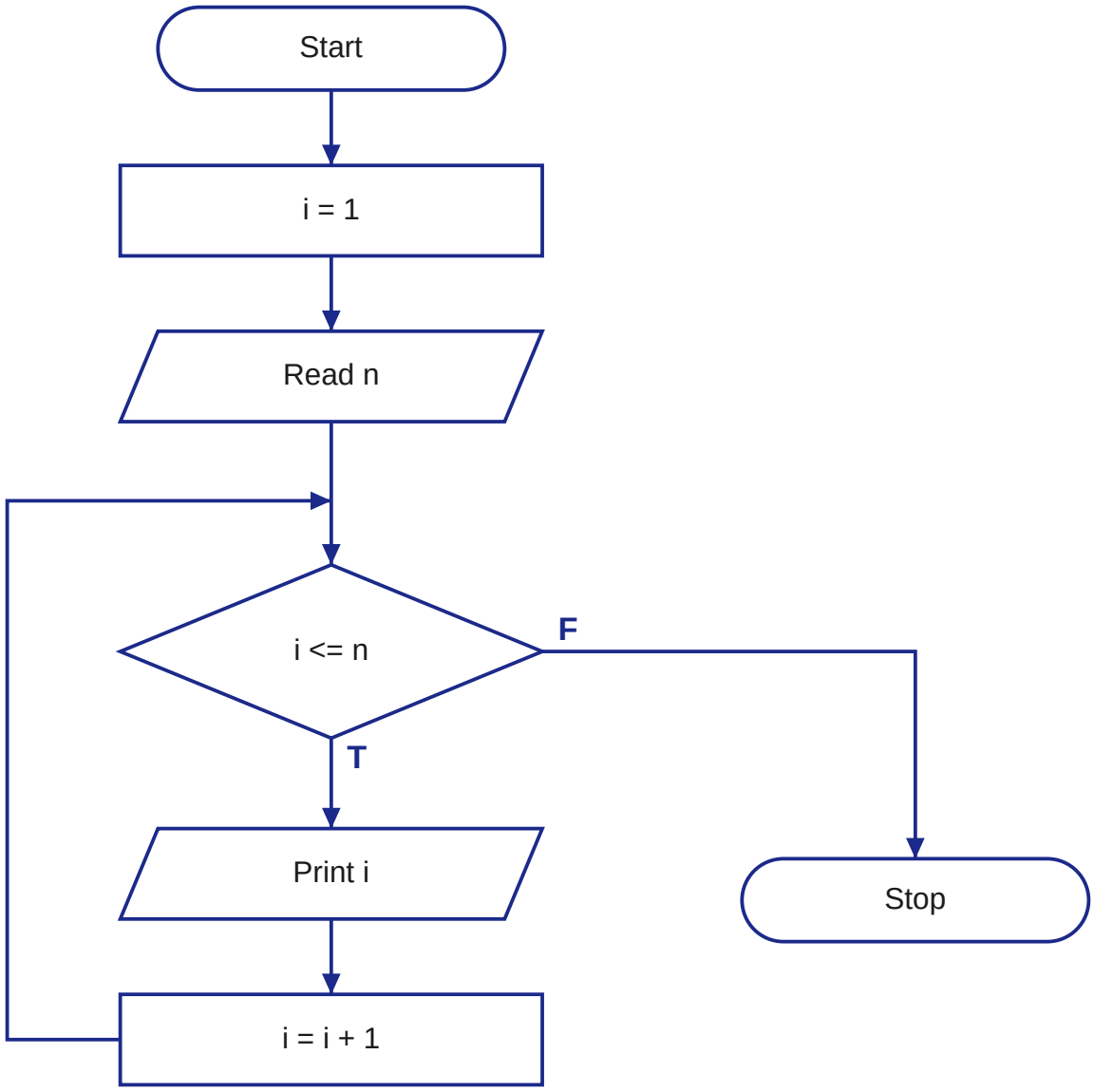


Algorithm

Step 1: Start
Step 2: Read a, b
Step 3: Check a > b
if true goto step 4
if false goto step 5
Step 4: Print "a is largest", goto step 6
Step 5: Print "b is largest"
Step 6: Stop

Example
Input: a = 5, b = 3
Output: a is largest
(If both are equal, either one is the largest.)

Q2. Print 1 to n

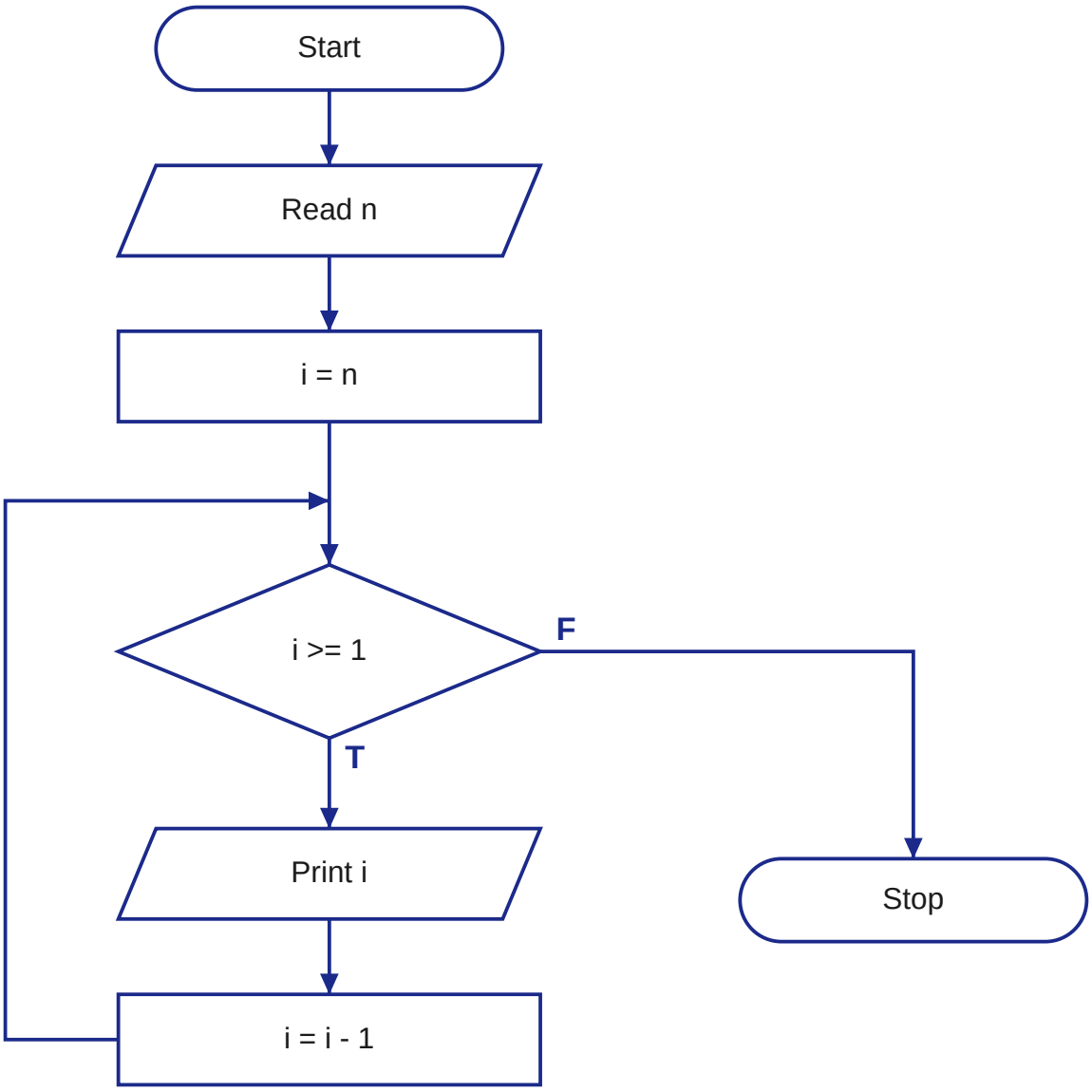


Algorithm

Step 1: Start
Step 2: Initialize i = 1
Step 3: Read n
Step 4: Check i <= n
if true goto step 5
if false goto step 7
Step 5: Print i
Step 6: i = i + 1, goto step 4
Step 7: Stop

Example
Input: n = 5
Output: 1 2 3 4 5

Q3. Print n to 1

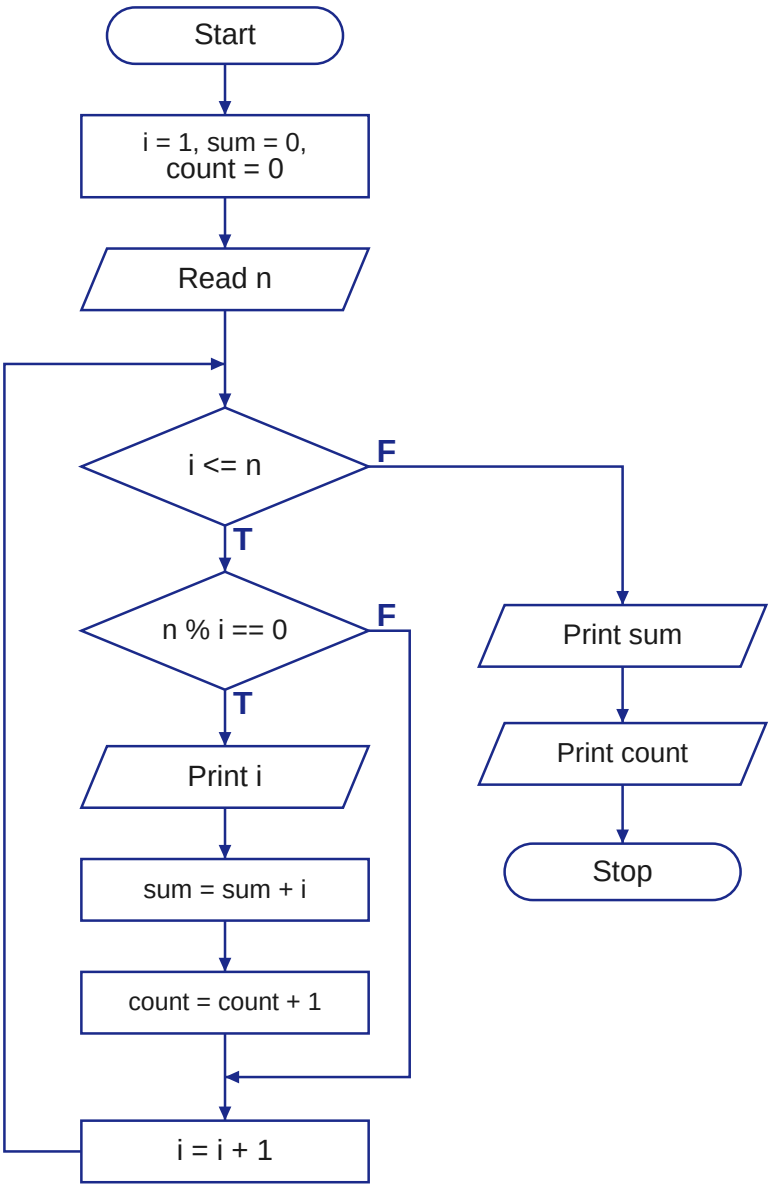


Algorithm

Step 1: Start
Step 2: Read n
Step 3: Initialize i = n
Step 4: Check i >= 1
if true goto step 5
if false goto step 7
Step 5: Print i
Step 6: i = i - 1, goto step 4
Step 7: Stop

Example
Input: n = 5
Output: 5 4 3 2 1

Q4. Print factors of a number, sum of factors, count

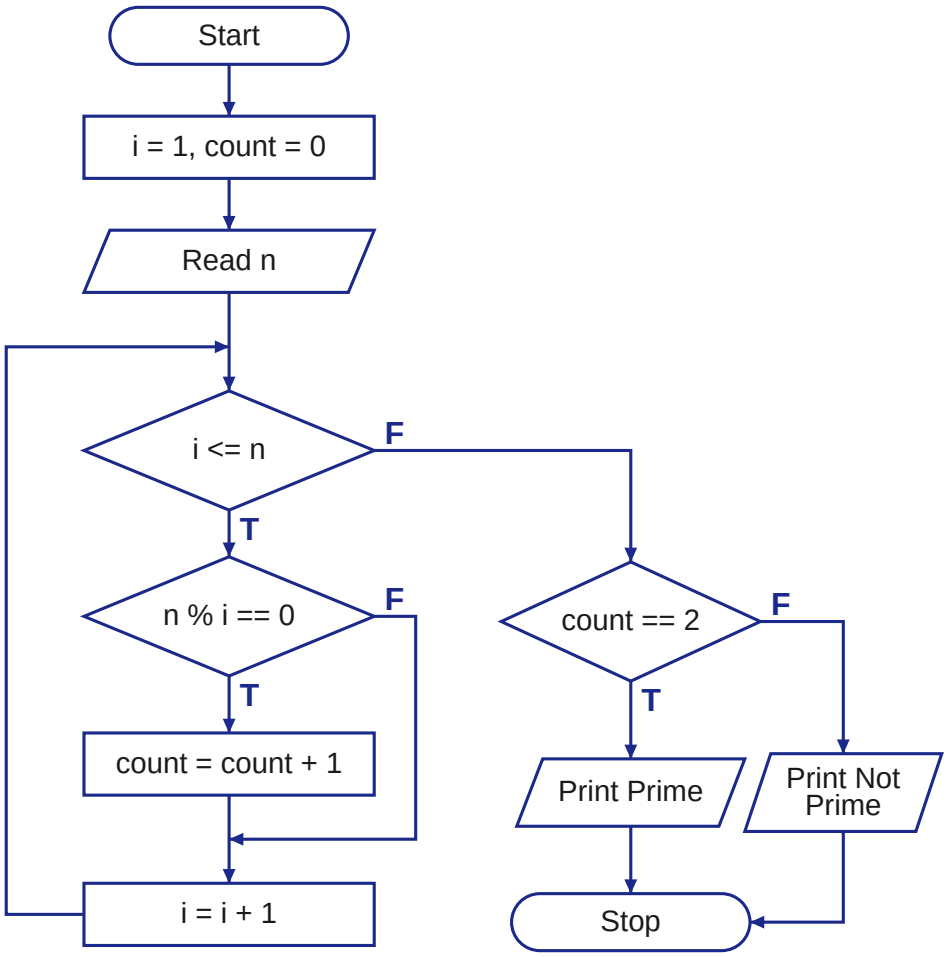


Algorithm

Step 1: Start
Step 2: Initialize i = 1, sum = 0, count = 0
Step 3: Read n
Step 4: Check i <= n
if true goto step 5
if false goto step 10
Step 5: Check n % i == 0
if true goto step 6
if false goto step 9
Step 6: Print i
Step 7: sum = sum + i
Step 8: count = count + 1
Step 9: i = i + 1, goto step 4
Step 10: Print sum
Step 11: Print count
Step 12: Stop

Example
Input: n = 6
Factors: 1 2 3 6
sum = 12, count = 4

Q5. Check prime or not

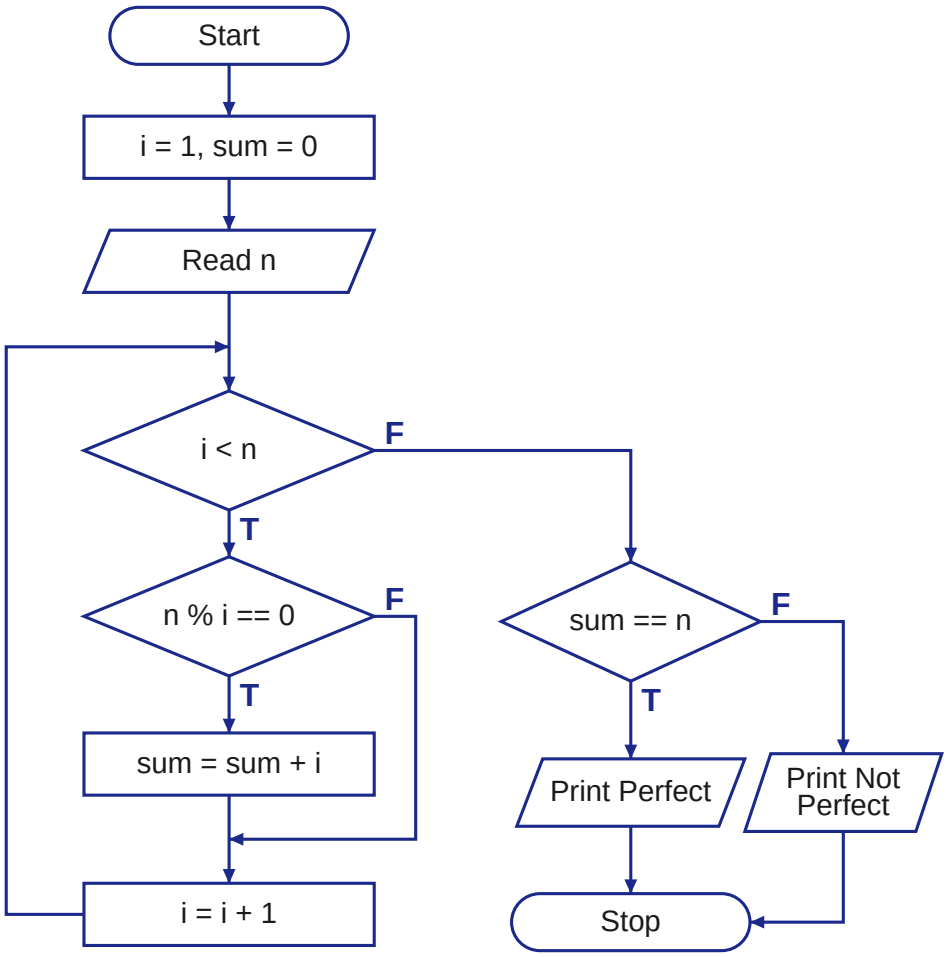


Algorithm

Step 1: Start
Step 2: Initialize i = 1, count = 0
Step 3: Read n
Step 4: Check i <= n
    if true goto step 5
    if false goto step 8
Step 5: Check n % i == 0
    if true goto step 6
    if false goto step 7
Step 6: count = count + 1
Step 7: i = i + 1, goto step 4
Step 8: Check count == 2
    if true goto step 9
    if false goto step 10
Step 9: Print "Prime", goto step 11
Step 10: Print "Not Prime"
Step 11: Stop

Example
Input: n = 7
Factors of 7 are 1 and 7, so count = 2
Output: Prime

Q6. Check perfect or not

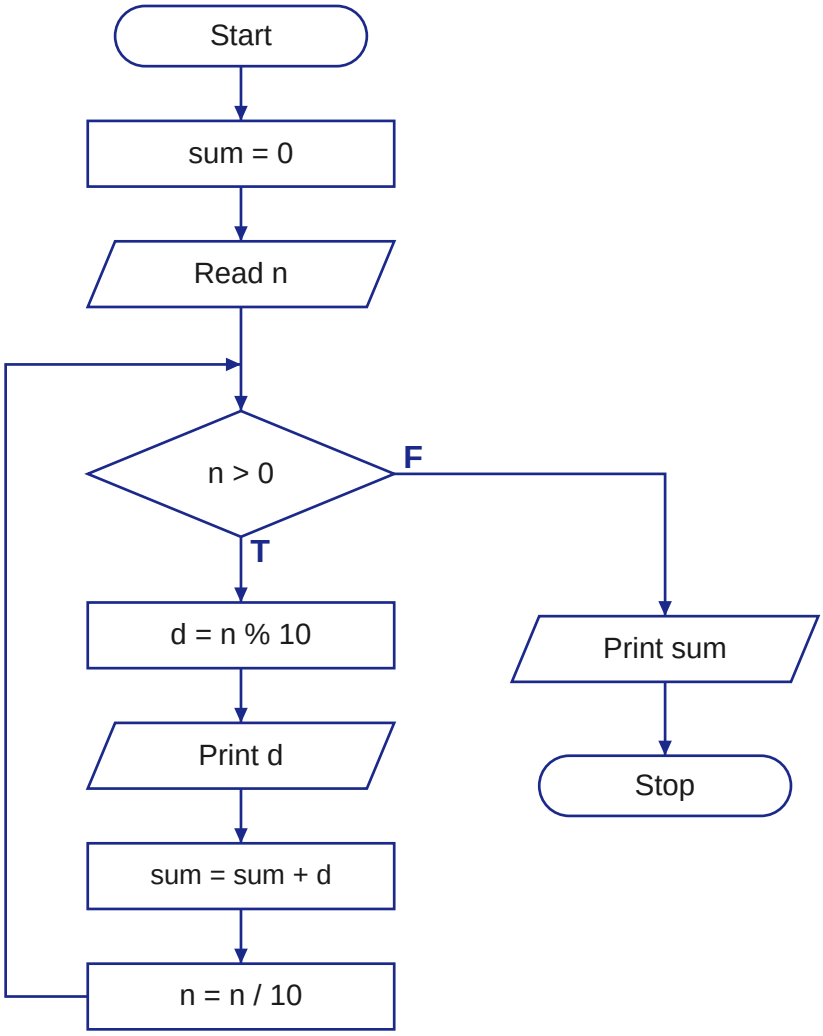


Algorithm

Step 1: Start
Step 2: Initialize i = 1, sum = 0
Step 3: Read n
Step 4: Check i < n
  if true goto step 5
  if false goto step 8
Step 5: Check n % i == 0
  if true goto step 6
  if false goto step 7
Step 6: sum = sum + i
Step 7: i = i + 1, goto step 4
Step 8: Check sum == n
  if true goto step 9
  if false goto step 10
Step 9: Print "Perfect", goto step 11
Step 10: Print "Not Perfect"
Step 11: Stop

Example
Input: n = 6
Factors less than 6: 1, 2, 3
sum = 1+2+3 = 6, Output: Perfect

Q7. Digits of a number, sum of digits

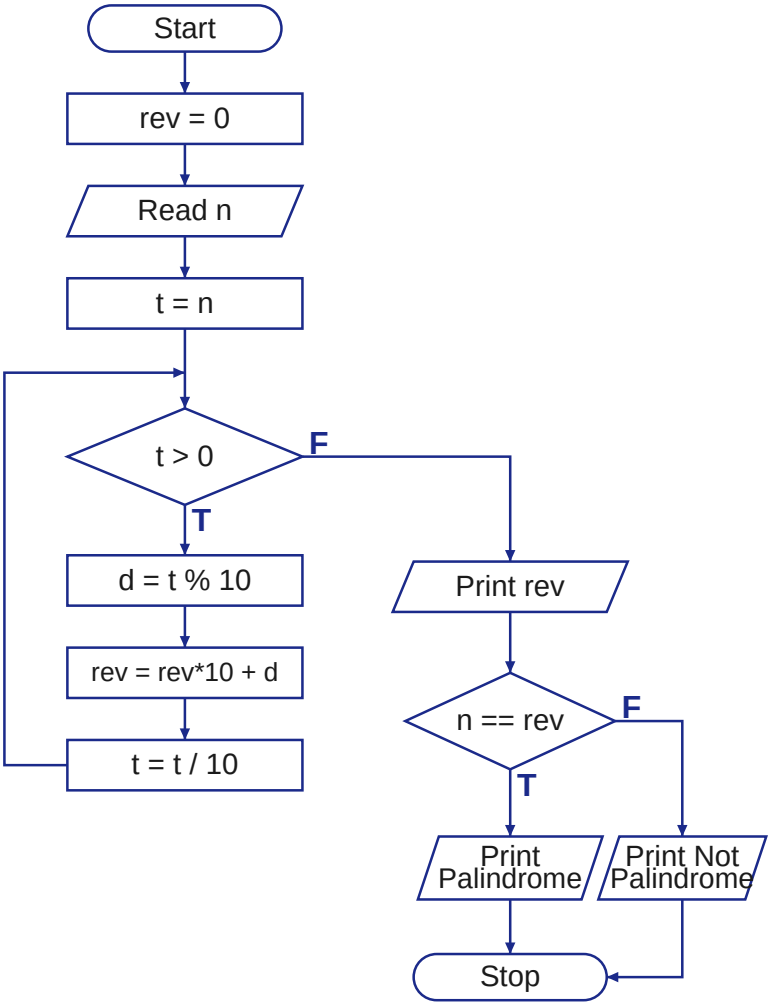


Algorithm

Step 1: Start
Step 2: Initialize sum = 0
Step 3: Read n
Step 4: Check n > 0
if true goto step 5
if false goto step 9
Step 5: d = n % 10
Step 6: Print d
Step 7: sum = sum + d
Step 8: n = n / 10, goto step 4
Step 9: Print sum
Step 10: Stop

Example
Input: n = 253
Digits printed: 3 5 2 (last digit first)
Sum of digits = 10
(n / 10 means integer division: 253 / 10 = 25)

Q8. Print reverse and check palindrome or not

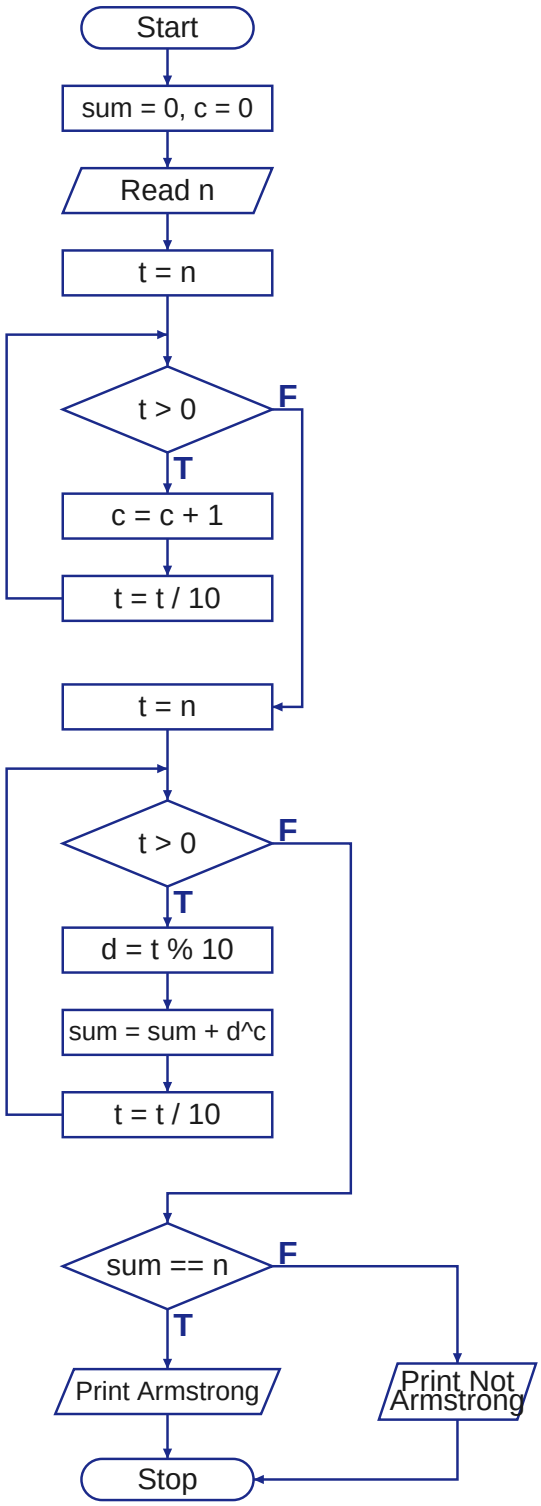


Algorithm

Step 1: Start
Step 2: Initialize rev = 0
Step 3: Read n
Step 4: t = n
Step 5: Check t > 0
if true goto step 6
if false goto step 9
Step 6: d = t % 10
Step 7: rev = rev * 10 + d
Step 8: t = t / 10, goto step 5
Step 9: Print rev
Step 10: Check n == rev
if true goto step 11
if false goto step 12
Step 11: Print "Palindrome", goto step 13
Step 12: Print "Not Palindrome"
Step 13: Stop

Example
Input: n = 121
rev = 121, so n == rev
Output: 121, Palindrome

Q9. Check Armstrong or not

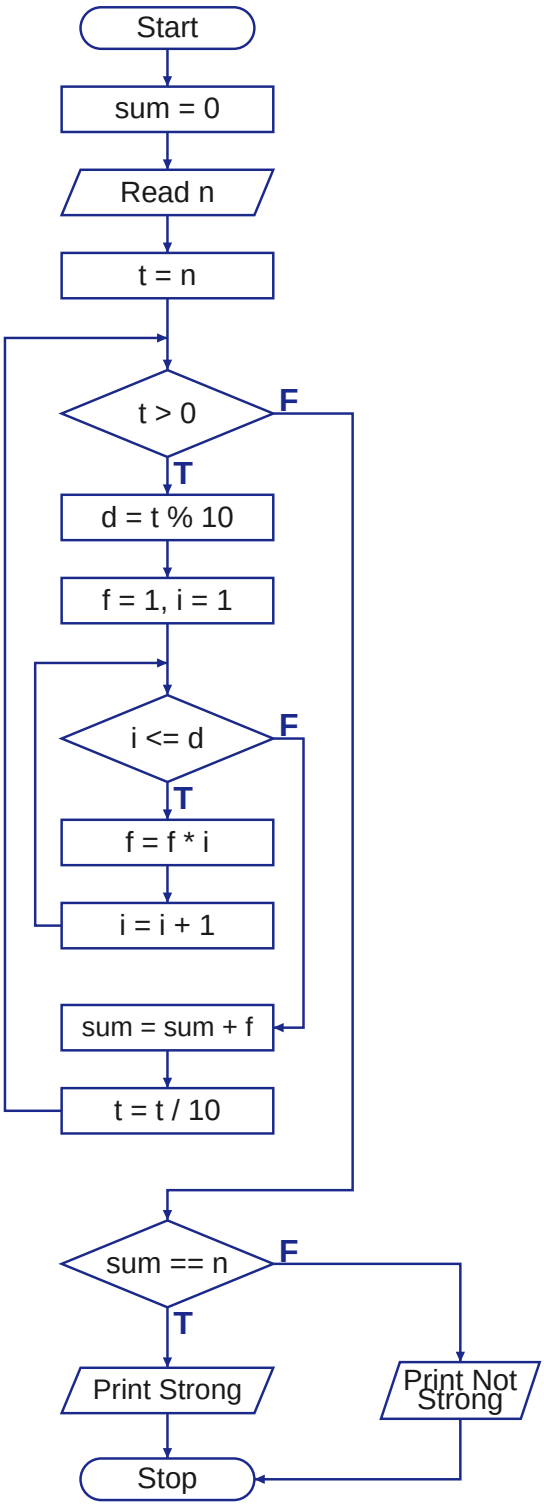


Algorithm

Step 1: Start
Step 2: Initialize sum = 0, c = 0
Step 3: Read n
Step 4: t = n
Step 5: Check t > 0
  if true goto step 6
  if false goto step 8
Step 6: c = c + 1 (counts digits)
Step 7: t = t / 10, goto step 5
Step 8: t = n
Step 9: Check t > 0
  if true goto step 10
  if false goto step 13
Step 10: d = t % 10
Step 11: sum = sum + d^c
Step 12: t = t / 10, goto step 9
Step 13: Check sum == n
  if true goto step 14
  if false goto step 15
Step 14: Print "Armstrong", goto step 16
Step 15: Print "Not Armstrong"
Step 16: Stop

Example
Input: n = 153, so c = 3 digits
1^3 + 5^3 + 3^3 = 1+125+27 = 153
Output: Armstrong
(d^c means d raised to the power c)

Q10. Check strong or not



Algorithm

Step 1: Start
Step 2: Initialize sum = 0
Step 3: Read n
Step 4: t = n
Step 5: Check t > 0
if true goto step 6
if false goto step 13
Step 6: d = t % 10
Step 7: f = 1, i = 1
Step 8: Check i <= d
if true goto step 9
if false goto step 11
Step 9: f = f * i
Step 10: i = i + 1, goto step 8
Step 11: sum = sum + f
Step 12: t = t / 10, goto step 5
Step 13: Check sum == n
if true goto step 14
if false goto step 15
Step 14: Print "Strong", goto step 16
Step 15: Print "Not Strong"
Step 16: Stop

Example
Input: n = 145
1! + 4! + 5! = 1+24+120 = 145
Output: Strong
(f is the factorial of each digit d)